

PROGRAMACIÓN IV

Trabajo Práctico

PARTE A

Migrar el “Gestor de Productos” de almacenamiento en memoria a PostgreSQL utilizando SQLAlchemy, manteniendo todas las validaciones existentes y demostrando comprensión de la arquitectura modular FastAPI con persistencia de datos.

MARCO TEORICO

Concepto	Aplicación en el proyecto
FastAPI	Framework moderno de Python para crear APIs RESTful con validación automática de datos y documentación interactiva.
SQLModel	Biblioteca que combina SQLAlchemy ORM y Pydantic, permitiendo definir modelos de datos que sirven tanto para base de datos como para validación.
Pydantic	Biblioteca de validación de datos que define schemas para request/response bodies con validación automática de tipos y constraints.
Arquitectura en capas	Patrón de diseño que separa responsabilidades en capas: Router (HTTP), Service (lógica de negocio), Repository (acceso a datos), Model (entidades).
PostgreSQL	Sistema de gestión de base de datos relacional de código abierto, que reemplaza el almacenamiento en memoria con persistencia real.
Alembic	Herramienta de migraciones que gestiona cambios en el esquema de la base de datos de forma versionada y reversible.

CASO PRÁCTICO

Dado el Trabajo Práctico de la unidad 4(FastApi):

- a) Migrar el proyecto y añadir PostgreSQL
- b) Probar endPoints (Postman/restClient)
- c) Agregar al archivo .zip los json de prueba
- d) Verificar en Swagger UI
 - Acceder a <http://localhost:8000/docs>
 - Probar cada endpoint interactivamente
 - Verificar schemas en la documentación

CONCLUSIONES ESPERADAS

Al finalizar el trabajo práctico, el estudiante debe demostrar:

- **Arquitectura modular:** Separación clara entre capas (Router → Service → Schema → Model)
- **Persistencia con SQLAlchemy:** Uso correcto de ORM y sesiones de base de datos
- **Validaciones robustas:** Validaciones a nivel de Pydantic (schemas) y reglas de negocio (service)
- **Manejo de errores:** Respuestas HTTP apropiadas para casos de éxito y error
- **Buenas prácticas:** Código modular, reutilizable, siguiendo principios de Clean Architecture

RECURSOS ADICIONALES

Documentación oficial:

- FastAPI: <https://fastapi.tiangolo.com/>
- SQLAlchemy: <https://sqlmodel.tiangolo.com/>
- Alembic: <https://alembic.sqlalchemy.org/>
- Pydantic: <https://docs.pydantic.dev/>

PARTE B

Crear una pantalla de Productos utilizando React con TypeScript, aplicando componentización, tipado estático y estilos con Tailwind CSS. No se requiere funcionalidad — solo maquetado.

MARCO TEÓRICO

Concepto	Aplicación en el proyecto
React	Biblioteca de JavaScript para construir interfaces de usuario mediante componentes reutilizables que describen cómo se ve la UI en cada momento.
TypeScript	Superset tipado de JavaScript que permite definir tipos estáticos, detectar errores en tiempo de compilación y mejorar la legibilidad del código.
Vite	Herramienta de build moderna que inicializa proyectos React con TypeScript de forma rápida, reemplazando a Create React App con mejor rendimiento.
Componentes	Funciones que retornan JSX y encapsulan una parte de la interfaz. Cada componente tiene una responsabilidad específica y puede recibir datos por props.
Props	Mecanismo para pasar datos de un componente padre a un componente hijo. En TypeScript se tipan con interfaces para garantizar la estructura correcta.
Interface	Construcción de TypeScript que define la forma de un objeto. Se usa para tipar los datos del dominio (ej: Producto) y los props de cada componente.
Tailwind CSS	Framework de CSS utilitario que aplica estilos directamente en el JSX mediante clases predefinidas, sin necesidad de escribir archivos CSS separados.
TSX	Extensión de sintaxis que permite escribir HTML dentro de JavaScript/TypeScript. Los archivos .tsx son componentes React con tipado TypeScript.
FastAPI	Framework Python de alto rendimiento para construir APIs REST. Permite definir endpoints, validar datos con Pydantic y exponer documentación automática en /docs.
Pydantic	Librería de validación de datos usada por FastAPI para definir modelos (schemas) con tipado estricto.
CORS	Mecanismo que permite al frontend (localhost:5173) comunicarse con el backend (localhost:8000) en dominios distintos.

CASO PRÁCTICO

Desarrollar el proyecto siguiendo los pasos indicados:

- e) Configurar el proyecto con Vite y TypeScript

Ejecutar en terminal:

```
pnpm create vite@latest tp-productos -- --template react-ts
pnpm install
```

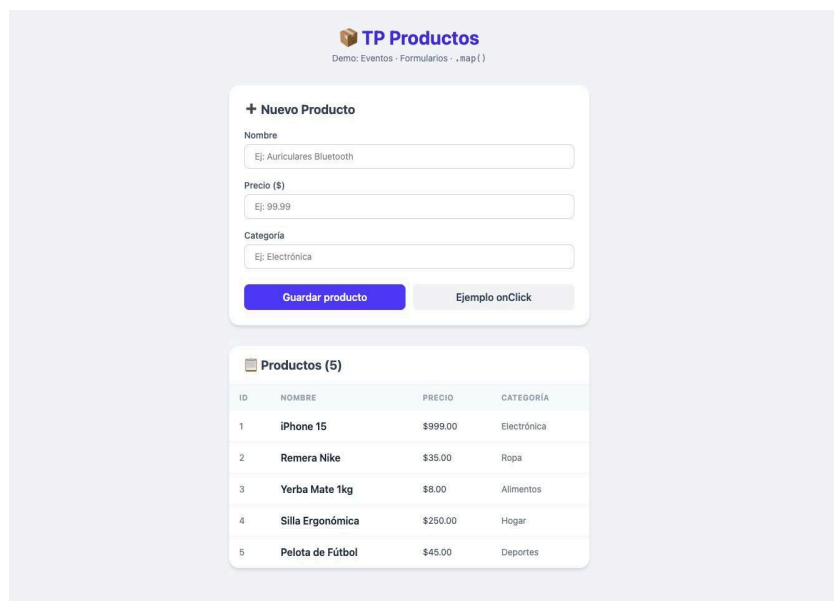
- f) Instalar y configurar Tailwind CSS según la guía oficial para Vite
g) Definir el tipado en src/types/producto.ts

```
export interface Producto {
  id: number
  nombre: string
  descripcion: string
  precio: number
}
```

- h) Crear los componentes en src/components/ (extensión .tsx):
- Navbar.tsx — barra de navegación con el nombre de la app
 - ProductoCard.tsx — tarjeta con los datos de un producto (props tipados)
 - ProductoList.tsx — recibe Producto[] por props y renderiza varias tarjetas
 - ProductoForm.tsx — formulario de alta con inputs (sin funcionalidad)
 - Footer.tsx — pie de página simple
- i) Maquetar en App.tsx con al menos 3 productos hardcodeados
j) Verificar que el proyecto corre sin errores de TypeScript

Ejecutar: pnpm dev y confirmar que no hay errores en consola ni en el compilador.

EJEMPLO DE PANTALLA



HISTORIAS DE USUARIO

Las siguientes historias de usuario describen, desde la perspectiva del actor que opera el sistema, el comportamiento esperado de la aplicación. Cada historia sigue el formato "Como... quiero... para..." y se

acompania de sus criterios de aceptacion, que constituyen la condicion verificable de que la historia fue implementada correctamente.

ID	Historia de usuario y criterios de aceptacion	Canal de verificacion
HU-01	Migrar backend de memoria a PostgreSQL con SQLAlchemy Como administrador del sistema, quiero migrar el Gestor de Productos de almacenamiento en memoria a PostgreSQL utilizando SQLAlchemy, para que los datos persistan entre reinicios del servidor. Criterios de aceptacion: • El proyecto FastAPI existente se migra de lista en memoria a base de datos PostgreSQL usando SQLAlchemy. • Los modelos SQLAlchemy reemplazan las estructuras en memoria manteniendo la misma validacion Pydantic. • Las operaciones CRUD (GET, POST, PUT, DELETE) funcionan contra la base de datos.. • Los endpoints se prueban con Postman/REST Client y se entregan archivos .json de prueba.	Backend (FastAPI + SQLAlchemy)
HU-02	Validar datos con Pydantic y reglas de negocio en service Como administrador del sistema, quiero que el sistema valide los datos de entrada con Pydantic y aplique reglas de negocio en la capa de servicio, para garantizar la integridad del catalogo. Criterios de aceptacion: • Los schemas Pydantic definen validacion de tipos, valores minimos y obligatoriedad. • Las reglas de negocio se implementan en la capa de servicio (no en router ni en modelo). • Errores de validacion devuelven 422 con detalle de los campos invalidos. • Conflictos de negocio (nombre duplicado, etc.) devuelven 409 con mensaje descriptivo.	Backend (Pydantic + Service)
HU-03	Documentar API con Swagger UI Como desarrollador, quiero acceder a la documentacion automatica de la API en /docs, para probar los endpoints interactivamente y conocer los schemas. Criterios de aceptacion: • FastAPI genera documentacion automatica en /docs y /redoc. • Cada endpoint muestra su metodo, ruta, parametros y schemas de request/response. • Los schemas Pydantic se reflejan en la documentacion. • Se puede probar cada endpoint desde Swagger UI sin herramientas externas.	Backend (FastAPI /docs)

HU-04	Crear pantalla de productos con React y TypeScript Como usuario, quiero ver una pantalla con productos maquetados utilizando React con TypeScript y Tailwind CSS, para visualizar el catalogo de forma atractiva. Criterios de aceptacion: • El proyecto se inicializa con Vite + React + TypeScript. • Se define la interface Producto en src/types/producto.ts con id, nombre, descripcion y precio. • Los componentes Navbar, ProductoCard, ProductoList, ProductoForm y Footer estan creados y tipados. • ProductoList recibe Producto[] por props y renderiza multiple ProductoCard. • App.tsx maqueta al menos 3 productos hardcodeados con Tailwind CSS. • El proyecto compila sin errores de TypeScript con pnpm dev.	Frontend (React + TypeScript)
HU-05	Aplicar estilos con Tailwind CSS Como usuario, quiero que la pantalla de productos tenga un diseno moderno y responsive usando Tailwind CSS, para una experiencia visual agradable. Criterios de aceptacion: • Tailwind CSS se configura segun la guia oficial para Vite. • Los estilos se aplican con clases utilitarias directamente en el JSX. • El layout usa grid/flex para ser responsive. • La paleta de colores y espaciado es consistente en toda la pantalla.	Frontend (Tailwind CSS)
HU-06	Estructurar proyecto con componentes puros Como desarrollador, quiero que la aplicacion React use solo componentes funcionales puros sin hooks, para mantenerla simple y enfocada en maquetado. Criterios de aceptacion: • Todos los componentes son funciones sin useState ni useEffect. • Los datos se pasan mediante props tipados desde App.tsx. • No hay llamadas a API ni logica de estado. • La estructura de carpetas sigue components/ y types/. • El README documenta la estructura del proyecto.	Frontend (Estructura)

REGLAS DE NEGOCIO

Las reglas de negocio definen las restricciones que el sistema debe hacer cumplir para mantener la integridad de los datos. Son responsabilidad del backend (capa de validacion con Pydantic y logica de los servicios) y complementan las historias de usuario: una historia describe que quiere lograr el usuario, mientras que una regla de negocio establece que esta permitido y que debe rechazarse.

ID	Regla de negocio	Entidad / Verificacion
RN-01	Los modelos SQLAlchemy se definen en archivos separados con tipos Python estrictos y validacion Pydantic.	Modelo / esquema SQLAlchemy
RN-02	El nombre del producto es obligatorio (min_length=1) y no puede quedar vacio ni contener solo espacios.	Producto / 422
RN-03	El precio del producto debe ser un numero mayor o igual a 0.	Producto / 422
RN-04	No se puede consultar, actualizar ni eliminar un producto cuyo id no exista en la base de datos.	Producto / 404 Not Found
RN-05	Toda operacion de escritura (POST, PUT, DELETE) persiste los cambios en PostgreSQL mediante sesion SQLAlchemy.	Base de datos / SQLAlchemy Session
RN-06	Los datos migrados de memoria a PostgreSQL deben mantener la misma estructura y validacion.	Migracion / consistencia
RN-07	La interface Producto en TypeScript debe reflejar los campos del modelo del backend (id, nombre, descripcion, precio).	Frontend / interface Producto
RN-08	Todos los componentes .tsx deben tener props tipados con TypeScript; no se permite el uso de any.	Frontend / tipado
RN-09	Los componentes son funcionales puros: no deben usar useState, useEffect ni ningun hook de React.	Frontend / componentes puros
RN-10	La estructura de carpetas del frontend debe incluir components/ y types/ como minimo.	Frontend / estructura
RN-11	Los archivos de prueba (.json de Postman o .http de REST Client) deben demostrar cada endpoint del CRUD.	Backend / testing
RN-12	Al descomprimir el proyecto, npm install + npm dev debe funcionar sin errores.	Frontend / entregable
RN-13	El proyecto backend debe incluir requirements.txt con fastapi, uvicorn, sqlalchemy y sqlalchemy.	Backend / dependencias

CONCLUSIONES ESPERADAS

Al finalizar el trabajo práctico, el estudiante debe demostrar:

- **Componentización correcta:** Separación clara de responsabilidades en componentes reutilizables
- **Tipado con TypeScript:** Uso de interfaces para Producto y props tipados en cada componente (.tsx)
- **Tailwind CSS:** Estilos aplicados con clases utilitarias, layout responsive con grid y flex
- **Estructura de proyecto:** Organización en carpetas components/ y types/ siguiendo buenas prácticas

- **Sin hooks:** El maquetado no requiere useState ni useEffect — solo componentes funcionales puros

MODALIDAD DE ENTREGA

- Eliminar la carpeta node_modules y venv antes de comprimir
- Ambos proyectos comprimidos en un unico archivo .zip
- Al descomprimir: npm install + npm run dev debe funcionar sin errores

RECURSOS ADICIONALES

Documentación oficial:

- React: <https://react.dev/>
- TypeScript: <https://www.typescriptlang.org/docs/>
- Tailwind CSS: <https://tailwindcss.com/docs/>
- Vite: <https://vitejs.dev/>